

Try with Simpler – An Evaluation of Improved Principal Component Analysis in Log-based Anomaly Detection

LIN YANG, College of Intelligence and Computing, Tianjin University, China

JUNJIE CHEN*, College of Intelligence and Computing, Tianjin University, China

SHUTAO GAO, Tianjin International Engineering Institute, the School of Future Technology, Tianjin University, China

ZHIHAO GONG, College of Intelligence and Computing, Tianjin University, China

HONGYU ZHANG, Chongqing University, China

YUE KANG, College of Intelligence and Computing, Tianjin University, China

HUAAN LI, College of Intelligence and Computing, Tianjin University, China

With the rapid development of deep learning (DL), the recent trend of log-based anomaly detection focuses on extracting semantic information from log events (i.e., templates of log messages) and designing more advanced DL models for anomaly detection. Indeed, the effectiveness of log-based anomaly detection can be improved, but these DL-based techniques further suffer from the limitations of heavier dependency on training data (such as data quality or data labels) and higher costs in time and resources due to the complexity and scale of DL models, which hinder their practical use. On the contrary, the techniques based on traditional machine learning or data mining algorithms are less dependent on training data and more efficient, but produce worse effectiveness than DL-based techniques which is mainly caused by the problem of unseen log events (some log events in incoming log messages are unseen in training data) confirmed by our motivating study. Intuitively, if we can improve the effectiveness of traditional techniques to be comparable with advanced DL-based techniques, log-based anomaly detection can be more practical. Indeed, an existing study in the other area (i.e., linking questions posted on Stack Overflow) has pointed out that traditional techniques with some optimizations can indeed achieve comparable effectiveness with the state-of-the-art DL-based technique, indicating the feasibility of enhancing traditional log-based anomaly detection techniques to some degree.

Inspired by the idea of “try-with-simpler”, we conducted the first empirical study to explore the potential of improving traditional techniques for more practical log-based anomaly detection. In this work, we optimized the traditional unsupervised PCA (Principal Component Analysis) technique by incorporating a lightweight semantic-based log representation in it, called *SemPCA*, and conducted an extensive study to investigate the potential of *SemPCA* for more practical log-based anomaly detection. By comparing seven log-based anomaly detection techniques (including four DL-based techniques, two traditional techniques, and *SemPCA*) on both public and industrial datasets, our results show that *SemPCA* achieves comparable effectiveness as advanced supervised/semi-supervised DL-based techniques while being much more stable under insufficient training

*Corresponding author.

Authors' addresses: Lin Yang, linyang@tju.edu.cn, College of Intelligence and Computing, Tianjin University, 135th, Yaguan Road, Tianjin, Tianjin, China, 30072; Junjie Chen, College of Intelligence and Computing, Tianjin University, China, junjiechen@tju.edu.cn; Shutao Gao, Tianjin International Engineering Institute, the School of Future Technology, Tianjin University, China, gaoshutao@tju.edu.cn; Zhihao Gong, College of Intelligence and Computing, Tianjin University, China, gongzhihao@tju.edu.cn; Hongyu Zhang, Chongqing University, China, hyzhang@cqu.edu.cn; Yue Kang, College of Intelligence and Computing, Tianjin University, China, kangyue0327@tju.edu.cn; Huaan Li, College of Intelligence and Computing, Tianjin University, China, devon_zw@tju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 ACM.

ACM 1049-331X/2022/1-ART111

<https://doi.org/10.1145/1122445.1122456>

data and more efficient, demonstrating that the traditional technique can still excel after small but useful adaptation.

CCS Concepts: • **Software and its engineering** → **Maintaining software**.

Additional Key Words and Phrases: Anomaly Detection, Log Analysis, Deep Learning, Machine Learning, Empirical Study

ACM Reference Format:

Lin Yang, Junjie Chen, Shutao Gao, Zhihao Gong, Hongyu Zhang, Yue Kang, and Huaan Li. 2022. Try with Simpler – An Evaluation of Improved Principal Component Analysis in Log-based Anomaly Detection. *ACM Trans. Softw. Eng. Methodol.* 37, 4, Article 111 (January 2022), 28 pages. <https://doi.org/10.1145/1122445.1122456>

1 INTRODUCTION

With the scale and complexity of software increasing, logs have become more important for software maintenance [3, 6, 8, 9, 71, 80, 81]. Logs record system events and states of interest and are produced during system runtime. Developers can check software status, detect anomalies, and diagnose root causes by carefully inspecting the recorded logs. However, log data tend to be massive due to the large software scale [41, 50, 62], and thus manual inspection for logs is very difficult or even infeasible. Therefore, over the years, a large amount of work on automated log analysis has been conducted [11, 15, 25, 26, 29, 39, 70, 87, 88, 91–93].

Log-based anomaly detection is one of the most important tasks in automated log analysis, which aims to automatically detect software anomalies in time based on log data so as to reduce the loss caused by these anomalies [10, 17, 36, 40, 42, 43, 47, 69, 76, 79, 85]. The general process of log-based anomaly detection consists of two main steps: representing log events (i.e., templates of log messages) and log sequences (i.e., series of log events that record specific execution flows) as vectors, and then building an anomaly detection model via machine learning (ML) or data mining (DM) algorithms based on those log vectors. With the rapid development of deep learning (DL), the trend of log-based anomaly detection focuses on two aspects in recent years [17, 36, 43, 47, 74, 79, 89]: 1) more effectively extracting semantic information from log events for log representation, and 2) designing more advanced DL models for anomaly detection. The first aspect can help understand the semantics of log events, which facilitates to relieve the problem of unseen log events. This problem is mainly caused by software evolution (e.g., frequent modification of log statements in source code), resulting in that some incoming log events may not appear in training data [79, 84]. It is harmful to the effectiveness of log-based anomaly detection as demonstrated in the existing work [47, 79, 84]. The second aspect aims to build a more accurate model to learn the distinction between anomalies and normal cases.

Although the effectiveness of log-based anomaly detection is indeed improved, these advanced DL-based techniques further suffer from the limitations shared in the area of DL, which can largely hinder their practical use. First, they heavily rely on training data (such as data quality or data labels). This is because building more complex DL models has a higher requirement for the volume and distribution of training data (which has been confirmed by our study in Section 3.) Besides, supervised DL algorithms also rely on data labels. In practice, it is very hard to collect sufficient high-quality data with labels since manual labeling is costly (especially for tasks that require domain knowledge). Furthermore, new data has to be frequently collected due to frequent software evolution, which further aggravates the difficulty of training data collection. Second, these DL-based techniques tend to spend much time and computing resources on hyper-parameter tuning, model building, and prediction due to the complexity of DL models and the scale of training data. However, in practice, both efficient deployment of accurate anomaly detection models and real-time anomaly detection are desired [54].

On the contrary, traditional techniques adopt lightweight ML or DM algorithms (e.g., clustering algorithms [40] and PCA – Principal Component Analysis [76]) to build anomaly detection models. Hence, they are more efficient and less dependent on training data, especially for unsupervised algorithms (e.g., PCA [76]). However, as demonstrated in an existing study [29], these traditional log-based anomaly detection techniques have lower accuracy than the advanced DL-based techniques, which limits their practical use. Moreover, one major reason for the poor effectiveness of traditional techniques lies in that they do not consider the influence of unseen log events [79], which is also confirmed by our motivating study (to be presented in Section 3). Intuitively, if we can improve the effectiveness of traditional techniques and make them comparable with advanced DL-based ones in terms of effectiveness, log-based anomaly detection can be more practical. Previously, Fu et al. [21] proposed the idea of “try-with-simpler”, which pointed out that for the task of linking questions posted on Stack Overflow, traditional SVM (Support Vector Machine) with some optimizations can achieve comparable effectiveness with the state-of-the-art DL-based technique. This also indicates the feasibility of enhancing traditional log-based anomaly detection techniques to some degree. However, this direction remains unexplored in the area of log-based anomaly detection.

Inspired by the idea of “try-with-simpler” [21], we conducted the first empirical study to explore the potential of improving traditional techniques for more practical log-based anomaly detection. Here, we studied the typical traditional technique (i.e., PCA [76]) and optimized it by incorporating semantic-based log representation. For ease of presentation, we call the optimized PCA technique *SemPCA*. This is because (1) PCA is the most widely-studied *unsupervised* traditional technique in the literature [2, 7, 52, 57, 61, 65] (unsupervised algorithms tend to be more practical in terms of efficiency and data dependency [31]); (2) Ignoring the influence of unseen log events is a major cause to the poor effectiveness of traditional techniques, which can be relieved by extracting semantic information from log events for log representation [79, 84]. According to the existing studies [36, 47, 79, 84], the above-mentioned two limitations (i.e., depending on training data heavier and spending more cost on model construction) are mainly caused by the advanced DL models rather than the extraction of semantic information from log events. Therefore, incorporating semantic-based log representation into the traditional PCA technique does not make *SemPCA* further suffer from the two limitations. Specifically, we incorporated a simple and lightweight semantic-based log representation method into the traditional PCA technique in order to avoid incurring too much extra cost, which aggregates the word vectors in each log event by measuring the importance of each word through TF-IDF (Term Frequency - Inverse Document Frequency) [30, 59].

Based on *SemPCA*, our empirical study addressed three research questions to investigate whether improving the traditional technique with simple adaptation can make log-based anomaly detection more practical.

- **RQ1:** Can *SemPCA* achieve comparable effectiveness with the existing log-based anomaly detection techniques (especially the advanced DL-based techniques)?
- **RQ2:** Can *SemPCA* perform more stably under insufficient training data than the existing techniques?
- **RQ3:** How does *SemPCA* perform in terms of efficiency compared with the existing techniques?

In the study, we compared *SemPCA* with a set of widely-studied log-based anomaly detection techniques (including four DL-based ones and two traditional ones) on five datasets (including three widely-used public datasets – *HDFS* [77], *BGL* [50], and *Spirit* [62], and two industrial datasets). The two industrial datasets are collected from two real-world systems in two organizations (i.e.,

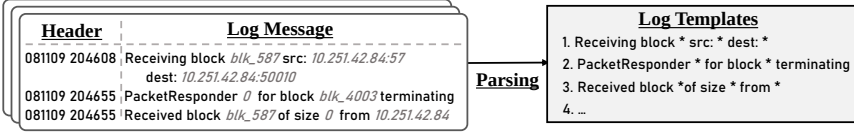


Fig. 1. An illustrating example for log terminology

the network center of our university and one influential motor corporation throughout the world¹), since *SemPCA* has been successfully applied to them due to its practical value.

Our results show that the optimized version of the unsupervised PCA technique (i.e., *SemPCA*) achieves comparable effectiveness with supervised/semi-supervised DL-based techniques, e.g., the average F1-score of *SemPCA* across all datasets is 0.959 while that of the most effective DL-based technique (i.e., the supervised LogRobust [84]) is 0.983. Moreover, *SemPCA* has been demonstrated to be much more efficient for model building and prediction, and more stable when facing insufficient training data. For example, for model building, *SemPCA* is at least 5,808X faster than the advanced DL-based techniques (e.g., LogRobust [84]). The results demonstrate the significant practicability of *SemPCA*, indicating that the typical traditional PCA technique can still excel in log-based anomaly detection after simple adaptation, also confirming the idea of “try-with-simpler” in our task.

To sum up, our work makes the following major contributions:

- We conduct a dedicated study, as the first attempt, to investigate whether traditional techniques through some optimizations can achieve comparable effectiveness with state-of-the-art DL-based techniques in log-based anomaly detection.
- We improve the traditional PCA technique by incorporating a lightweight semantic-based log representation method. By comparing with a set of advanced techniques on five datasets, it achieves comparable effectiveness while is more efficient and stable.
- We develop and release a toolbox that integrates the implementations of all the studied log-based anomaly detection techniques, in order to promote future research, practical use, and evaluation replication.

2 BACKGROUND

2.1 Log Terminology

Figure 1 shows an example of log data sampled from the public HDFS dataset [77], facilitating the illustration of log terminology. From this figure, a **log message** is a raw unstructured sentence generated during system runtime, which records software status of the time. A log message (e.g., *PacketResponder 0 for block blk_4003 terminating*) consists of a **log event** (e.g., *PacketResponder * for block * terminating*) and **log parameters** (e.g., *0, blk_4003*). The former is the template of a log message, which is programmed by developers as a log statement in source code. This is a constant part in a log message. The latter is a variable part that records some software attributes, such as IP address, file path, and block ID. In the area of log-based anomaly detection, log events can be automatically extracted from log messages via **log parsing**, which is the first step in log-based anomaly detection and has been widely studied over the years [14, 24, 48, 64, 67, 68, 93].

In practice, developers as well as log-based anomaly detection techniques tend to understand a sequence of log messages for checking software status. Here, we call a sequence of log events **log sequence**, which usually reflects a specific execution flow or the system status within a certain period. By checking the task ID of each log message or adopting some strategies (e.g., splitting

¹We hide the company name due to the company policy.

windows), we can know which log events belong to the same log sequence. Based on whether or not the log sequences indicate software anomalies, they can be classified as **anomalous log sequences** and **normal log sequences**.

2.2 Log-based Anomaly Detection Techniques

Due to the importance of log-based anomaly detection, a lot of techniques have been proposed in the literature [17, 40, 47, 76, 79, 84]. As presented in Section 1, these techniques share the same high-level process: 1) representing log events and log sequences as vectors, and 2) building an anomaly detection model through traditional or deep learning algorithms based on log vectors. With the rapid development of DL, the recent trend of log-based anomaly detection is to incorporate more advanced DL algorithms in order to improve the anomaly detection effectiveness. Here, according to whether they are based on traditional ML/DM algorithms or advanced DL algorithms, we classify them into two categories: *traditional techniques* and *DL-based techniques*. In the following, we introduce two traditional techniques and four DL-based techniques in detail because they are the most widely-studied in the existing studies [29, 47, 79]. Following these existing work, we also used them as the studied techniques in our study.

2.2.1 Traditional Log-based Anomaly Detection Techniques. The two most widely-studied traditional log-based anomaly detection techniques are PCA [76] and LogCluster [40].

PCA is one of the unsupervised techniques among these widely-studied techniques. It first represents each log sequence as an *Event Count Vector* by counting the occurrence of each log event in the log sequence. Then, it applies the PCA algorithm [2, 38] to identify principal components (which preserve data variation as much as possible) from event count vectors and projects those vectors to the space of principal components (called normal space). Based on the projections to the normal space, the projection of each vector to the abnormal space can be obtained accordingly. Since the projections of normal log sequences to the abnormal space should be less discrete, the anomalies can be detected through outlier detection in the abnormal space. Specifically, if the Square Prediction Error (SPE) [32] of a projection to the abnormal space is larger than a given threshold, the log sequence is regarded as an anomalous one.

LogCluster is a semi-automatic and supervised technique initially. It clusters training data into several groups, each of which is manually labeled as a normal or anomalous group. By determining which kind of groups an incoming log sequence belongs to, the anomaly detection result can be obtained. If an incoming log sequence does not belong to any of these groups, developers have to manually check it. To make LogCluster easy-to-study in practice, an adapted version of LogCluster is used in many existing studies [29, 36, 47, 79] (as well as our study), which is an automatic and semi-supervised technique. Specifically, it labels and clusters only normal training data to obtain normal groups. If an incoming log sequence does not belong to any of the groups, it is regarded as an anomaly. Before clustering through the Agglomerative Hierarchical Clustering algorithm [23], it first calculates the weight of each log event by an IDF-based weighting [44] and a contrast-based weighting [40], and then a log sequence is represented as a *Weighted Event Count Vector* based on the weights of the log events in the log sequence.

2.2.2 DL-based Log-based Anomaly Detection Techniques. The four most widely-studied DL-based techniques are DeepLog [17], LogRobust [84], LogAnomaly [47], and PLELog [79].

DeepLog is a widely-studied DL-based technique, which learn log patterns from the system's normal executions to build the anomaly detection model [17]. It first represents each log event as a unique ID and thus each log sequence is represented as a sequence of IDs (also called *Log Event ID Sequence*). Then it builds an LSTM (Long Short Term Memory [63]) model that can predict the next

ID according to the sequence of IDs occurring before the one to be predicted in a log sequence. If an ID is mispredicted in an incoming log sequence, this log sequence is regarded as an anomaly.

LogRobust [84] is a state-of-the-art supervised log-based anomaly detection technique. It is also the first to relieve the problem of unseen log events in log-based anomaly detection. It proposes the *TF-IDF-based Semantic Representation* method to represent each log event, which first adopts the pre-trained word vectors on the Common Crawl Corpus dataset using the Fast-Text toolkit [5, 34, 35] to represent each word in a log event as a vector, and then aggregates these word vectors as a log-event vector by measuring the importance of each word through TF-IDF [30, 59]. Based on these log-event vectors from training data, LogRobust builds a supervised model through attention-based Bi-directional LSTM [63], which can predict whether an incoming log sequence is normal or anomalous.

LogAnomaly extends DeepLog by improving log representation, which combines both *Event Count Vector* and *Synonyms-Antonyms-based Semantic Representation* [46]. The insight of synonyms-antonyms-based representation is to capture the semantic information of the involved synonyms and antonyms when representing a log event. This is because similar log events may also have very different semantics, e.g., “Interface * changed state to **down**” and “Interface * changed state to **up**”, and capturing the semantic information of synonyms and antonyms can help relieve this issue to obtain better log representation. This method requires to construct the set of synonyms and antonyms with the aid of operators based on domain knowledge. Then, it builds an embedding model based on dLCE [49] to represent a log event as a semantic vector. Based on the log representation, LogAnomaly learns both semantic patterns and quantity patterns from the system’s normal execution by two separate LSTM models. Similar to DeepLog, if a log event is mispredicted based on the log events occurring before this event in an incoming log sequence, this log sequence is regarded as an anomaly.

PLELog is the state-of-the-art semi-supervised log-based anomaly detection technique, which requires a set of labeled normal training data among the whole training set [79]. Its main contribution is to estimate the labels of the remaining unlabeled training data by clustering all the training data through the *HDBSCAN* algorithm [45]. According to the clustering results and the known normal log sequences, PLELog assigns a probabilistic label to each unlabeled log sequence in training data. PLELog also uses the *TF-IDF-based Semantic Representation* method to represent each log event for relieving the problem of unseen log events. In particular, it adopts GloVe, a pre-trained language model based on global vectors [53], to represent each word in a log event. Based on these vectors and estimated labels, it builds an model through attention-based Bi-directional GRU [13], which can predict whether an incoming log sequence is normal or anomalous.

2.2.3 Summary. For ease of understanding, Table 1 briefly summarizes the log-representation method and the algorithm for building the anomaly detection model used by each technique. In fact, both semantic-based log representation and advanced DL models contribute to the superiority of DL-based techniques over traditional techniques in terms of the anomaly detection effectiveness. Specifically, the former improves the effectiveness by relieving the problem of unseen log events as presented in Section 1. Moreover, it does not incur the above-mentioned limitations (i.e., depending on training data more heavily and spending more time on model construction) for DL-based techniques, since these semantic-based log representation methods directly use pre-trained language models to extract semantic information from log events [36, 51]. In other words, the above-mentioned limitations are mainly caused by the latter (i.e., advanced DL models), but the effectiveness improvement is brought by the two aspects. Therefore, this motivates a potential direction to optimize traditional techniques by incorporating semantic-based log representation,

Table 1. Summary of the Studied Log-based Anomaly Detection Techniques

Category	Technique	Log Representation	Anomaly Detection
Traditional	PCA	Event Count Vector	PCA
	LogCluster	Weighted Event Count Vector	Hierarchal Clustering
DL-based	DeepLog	Log Event ID Sequence	LSTM
	LogAnomaly	Event Count Vector& Synonyms-Antonyms-based Semantic Representation	Attention-based Bi-LSTM
	LogRobust	TF-IDF-based Semantic Representation	LSTM
	PLELog	TF-IDF-based Semantic Representation	HDBSCAN& Attention-based Bi-GRU

which is likely to improve the anomaly detection effectiveness but does not damage the advantages of high efficiency and weak data dependency.

3 MOTIVATING STUDY

In this section, we conducted a motivating study to quantitatively investigate the advantages and disadvantages of traditional techniques and DL-based techniques (mentioned in Section 1), which aims to motivate the potential of traditional techniques in log-based anomaly detection. We first compared DL-based techniques and traditional techniques in terms of effectiveness, efficiency, and stability (Section 3.1). As demonstrated in the existing study [21, 37], they are important factors affecting the practical use of log-based anomaly detection techniques. Then, we investigated whether the problem of unseen log events is an important factor affecting the effectiveness of traditional techniques (Section 3.2). Specifically, we used a state-of-the-art *supervised* technique, i.e., LogRobust [84], as the representative of DL-based techniques, and the *unsupervised* technique, i.e., PCA [76], as the representative of traditional techniques. They are significantly different, and such a strong contrast is more helpful in motivating our work. Here, we studied them on the public *Spirit* dataset [62], which was produced by the Spirit supercomputer at Sandia National Labs with a time span of 2.5 years. More details about this dataset can be found in Section 4.2).

3.1 LogRobust v.s. PCA

We evaluated the *effectiveness* of LogRobust and PCA on different sets of training data, which can help investigate their *stability*. Specifically, we split *Spirit* into training data, validation data, and test data in chronological order of log sequences with the ratio of 6:1:3. This can ensure that all log sequences in training data are produced before the log sequences in test data. In this study, we kept the validation and test data unchanged and constructed several training sets by randomly sampling 20% or 40% log data from the whole training data. We repeated the sampling process 10 times for each sampling ratio. In total, we obtained 20 training sets, i.e., 10 training sets with 20% of training data and 10 training sets with 40% of training data. Then, we built an anomaly detection model based on each training set using LogRobust and PCA respectively, and measured their effectiveness on the test data in terms of F1-score [56]. The hyper-parameter tuning process for each technique

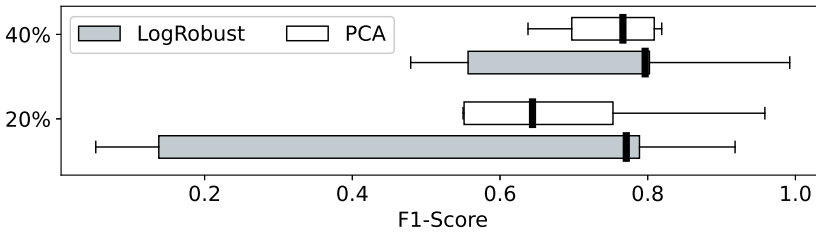


Fig. 2. F1-score under different sets of training data

in each training set is conducted via grid search on the same validation data following the practice in DL [58].

Figure 2 shows F1-score of LogRobust and PCA, where each box shows the overall effectiveness of each technique across the 10 training sets with 20% or 40% of training data. From this figure, the medium F1-score of LogRobust is larger than that of PCA regardless of 20% or 40% of training data, demonstrating the effectiveness of such an advanced supervised DL-based technique. However, LogRobust performs more unstably than PCA under each studied amount of training data, indicating that the former relies more heavily on the training data. Also, LogRobust becomes more stable with increasing amount of training data (from 20% to 40% in the study), indicating its heavy dependency on the training data volume. However, in practice, collecting sufficient high-quality training data is very challenging and costly, which largely hinders the practicability of such a DL-based technique.

We further measured the *time cost* spent on building an anomaly detection model based on the whole training data and predicting one incoming log sequence for LogRobust and PCA, respectively. The time cost spent on training LogRobust is 2,361.429 seconds, while that of PCA is only 0.983 second. Regarding the time cost spent on predicting incoming log sequences in the test data, LogRobust spends 1.704 seconds while PCA spends only 0.303 second. The results demonstrate the significant superiority of PCA over LogRobust in terms of training and prediction efficiency.

Finding 1: The supervised DL-based technique (i.e., LogRobust) outperforms the unsupervised traditional technique (i.e., PCA) in terms of effectiveness (the medium F1-score). However, the latter is much more efficient and stable.

3.2 Influence of unseen log events on PCA

According to Finding 1, if we can improve the effectiveness of PCA without obviously damaging its efficiency and stability, more practical log-based anomaly detection can be achieved. Hence, it is important to identify the important factor affecting the effectiveness of PCA. As demonstrated in the existing studies [17, 47, 79, 84], the problem of unseen log events is harmful to the effectiveness of log-based anomaly detection. Here, we conducted an experiment to investigate whether this problem is also an important factor that negatively affects the effectiveness of PCA. Specifically, we constructed three training sets by randomly sampling 20% training data from the whole training data. In particular, we ensured that the three training sets have different numbers of unseen log events with regard to the test data.

Based on each training set, we built an anomaly detection model using PCA and measured its F1-score on the same test data. The results are shown in Table 2. We found that with the number of unseen log events decreasing, the effectiveness of PCA becomes better in terms of F1-score, confirming the significant influence of the problem of unseen log events on PCA. With this finding, the effectiveness of PCA could be effectively improved by relieving the problem of unseen log

Table 2. F1-score of PCA under different numbers of unseen log events

#Unseen Events	404	311	283
F1-score	0.738	0.872	0.935

events. This further motivates the potential of such an unsupervised traditional technique for more practical log-based anomaly detection.

Finding 2: Relieving the problem of unseen log events may be a promising direction to improve the effectiveness of PCA.

4 METHODOLOGY

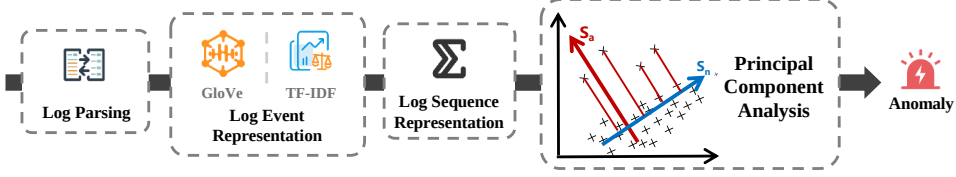
Based on the findings of the motivating study, we conducted an extensive study, as the first attempt, to explore the potential of traditional techniques for more practical log-based anomaly detection. In this section, we present the methodology of our empirical study, including the optimized version of the traditional PCA technique and the study design.

4.1 Studied techniques

As presented in Section 2.2, we evaluated six widely-studied log-based anomaly detection techniques, including two traditional techniques (i.e., PCA and LogCluster) and four DL-based techniques (i.e., DeepLog, LogAnomaly, LogRobust, and PLELog). Besides, the goal of our study is to explore whether traditional techniques through some adaptations can achieve comparable effectiveness with state-of-the-art DL-based techniques and thus make log-based anomaly detection more practical. Therefore, motivated by the findings in our motivating study in Section 3, we optimized the traditional PCA technique by incorporating semantic-based log representation. Please note that our work does not aim to propose a new technique, but explores the potential of existing traditional techniques through simple adaptations. Hence, we directly adopted the existing semantic-based log representation method for optimizing the traditional PCA technique. From Table 1, there are two semantic-based log representation methods (i.e., synonyms-antonyms-based [47] and TF-IDF-based semantic representation [84]) used in these widely-studied log-based anomaly detection techniques. Following the idea of “try-with-simpler”, we selected the lightweight TF-IDF-based semantic representation method (instead of Synonyms-Antonyms-based Semantic Representation) for optimizing the traditional PCA technique, since it is more light-weight without operator intervention and additional embedding model training. For ease of presentation, we call the optimized version of PCA *SemPCA*. In total, we studied seven log-based anomaly detection techniques in our study.

As the first attempt to explore the potential of traditional techniques for more practical log-based anomaly detection, we adopted the traditional PCA technique as the representative since it is a typical *unsupervised* technique and unsupervised algorithms tend to be more practical in terms of efficiency and data dependency [31]. Actually, other traditional techniques could be also combined with the semantic-based log representation method and thus achieve better effectiveness, which will be discussed in detail in Section 6.2. Since the former six techniques have been presented before, we introduce *SemPCA* in detail here. Figure 3 shows the workflow of *SemPCA*, also following the two main steps (i.e., log representation and anomaly detection model building).

4.1.1 Semantic-based Log Representation. *SemPCA* first extracts log events from log messages through log parsing and then represents log events as vectors. Specifically, *SemPCA* adopts the

Fig. 3. Workflow of *SemPCA*

widely-used log parser (i.e., Drain [24]) for log event extraction. For fair comparison, all the studied log-based anomaly detection techniques use the same log parser in our study, to eliminate the influence of different log parsers on anomaly detection.

Afterwards, *SemPCA* adopts the lightweight TF-IDF-based semantic representation method to represent each log event. In particular, *SemPCA* directly adopts the pre-trained word vectors on the Common Crawl Corpus dataset using the Fast-Text [34] toolkit for TF-IDF-based representation following the existing work [79, 84]. Specifically, the representation of a log event (denoted as e) is calculated by $V(e) = \sum_{i=1}^{L_e} (\omega_i \times \alpha_i)$, where ω_i is the word vector of the i^{th} word (denoted as w_i) in e , L_e is the number of words in e , and $\alpha_i = TF(w_i) \times IDF(w_i)$. Here, following the existing work [79], we adopts GloVe, a pre-trained language model based on global vectors [53], to represent each word in a log event as a word vector. TF (Term Frequency) measures how frequently a word w occurs in e , i.e., $TF(w) = \frac{\#w}{L_e}$, where $\#w$ is the number of the occurrence of w in e . IDF (Inverse Document Frequency) measures how common or rare a word w is in all the log events, i.e., $IDF(w) = \log(\frac{\#E}{\#E_w})$, where $\#E$ is the total number of log events and $\#E_w$ is the number of log events that contain w . Indeed, there are some more advanced methods than GloVe and TF-IDF respectively, but our work does not aim to evaluate all of them. If these simple methods can work well for improving the traditional PCA technique, we believe that more advanced ones can be helpful to further improve the effectiveness, which can be regarded as our future work. Also, in the future, we can investigate the effectiveness of other log representation methods (such as synonym-antonym-based representation) in improving traditional techniques.

4.1.2 Anomaly Detection Model Building. An anomaly detection model is constructed based on log sequences. Hence, after obtaining log event vectors, it requires to further represent a log sequence as a vector. Existing log sequence representation methods either simply aggregate log event vectors via addition or incorporate advanced DL models to represent the log sequences. Since the former is more lightweight, *SemPCA* adopts it for log sequence representation in order to keep the advantages of its high efficiency and weak data dependency. In the future, more log sequence representation methods can be explored.

Based on the log sequence vectors, *SemPCA* adopts the PCA algorithm to build an anomaly detection model. The core of PCA in anomaly detection is to build a **normal space** (denoted as S_n , and an **abnormal space** (denoted as S_a) as shown in the PCA step in Figure 3. Specifically, it projects each log sequence vector (denoted as v_i) into y_n^i by $y_n^i = PP^T v_i$, where $P = [e_1, e_2, \dots, e_k]$ and $e_1, e_2, \dots, e_k$ refer to the eigenvectors with the k largest eigenvalues. That also indicates that there are k principal components. We call y_n^i the projection of v_i to the normal space (S_n). Similarly, it also projects v_i to the abnormal space via $y_a^i = (I - PP^T)v_i$, where I refers to the standard identity matrix. Since the components corresponding to the abnormal space retain the minimal variance of the original data, *SemPCA* detects anomalies through outlier detection in the abnormal space (S_a).

Table 3. Basic information of datasets. The window size of the Fixed Window strategy refers to the number of log messages within each window, while the window size of the Fixed Time Window strategy refers to the time period within which all log messages are grouped as a log sequence.

Datasets	# Log Messages	Grouping Strategy	avg. Length	# Sequences			% Anomaly			# Events	# Unseen
				<i>train</i>	<i>val</i>	<i>test</i>	<i>train</i>	<i>val</i>	<i>test</i>		
<i>HDFS</i>	11,175,629	Session Window (block_id)	19	345,036	57,506	172,518	3.27	3.20	2.15	46	1
<i>BGL</i>	4,747,963	Session Window (node_id) + Fixed Window (size = 120)	55	51,346	8,557	25,673	52.96	53.21	17.75	431	40
<i>Spirit</i>	12,000,000	Fixed Window (size = 120)	120	60,000	10,000	30,000	25.74	25.39	29.98	1414	53
<i>NC</i>	334,567	Fixed Time Window (size = 5min)	13	15,441	2,573	7,721	21.43	6.86	2.19	23	12
<i>MC</i>	6,148,033	Fixed Time Window (size = 5min)	210	17,550	2,925	8,775	-	-	-	245	31

Following the existing work [76], *SemPCA* adopts Square Prediction Error (SPE) [18] to determine whether an incoming log sequence (denoted as v_j) is an outlier or not in the abnormal space by comparing with a given threshold θ (which is a hyper-parameter in *SemPCA*). Specifically, the SPE value for v_j is calculated by $\|y_a^j\|^2$. If the SPE value is larger than θ , v_j is predicted as an anomalous log sequence.

4.2 Datasets

In our study, we evaluated the studied log-based anomaly detection techniques on five datasets, including three public datasets (i.e., *HDFS* [77], *BGL* [50], *Spirit* [62]) that have been widely-used in the existing work [17, 36, 47, 79], as well as two industrial datasets from two large-scale distributed online systems in two different organizations, i.e., the network center of our university and one influential motor corporation throughout the world. We hide the company name and system names due to the confidential policy. For ease of presentation, we call the two industrial datasets *NC* and *MC*, respectively.

Table 3 shows the basic information of these datasets, including the number of log messages, the used method of log data grouping (i.e., log sequence construction by grouping log messages), the average length of a log sequence, the number of log sequences in the training/validation/test sets, the ratio of anomalous log sequences in the training/validation/test sets, the total number of log events, and the number of unseen log events in the test set. In the following, we will introduce each dataset in detail.

HDFS (Hadoop Distributed File System) was collected during the running of Hadoop-based MapReduce jobs on more than 2,000 Amazon’s EC2 nodes for 38.7 hours. It contains 11,175,629 log messages, which form 575,062 log sequences according to the *block_id* of each log message. Among them, 2.9% of log sequences are anomalous, which were manually labeled by Hadoop domain experts.

BGL (Blue Gene/L supercomputer) was produced by the Blue Gene/L supercomputer, which consists of 128K processors and was deployed at the Lawrence Livermore National Labs with a time span of 7 months. In total, it contains 4,747,963 log messages, among which 348,460 are anomalous. Following the existing work [17, 47, 79], we combined the fixed window strategy and the session window strategy, which apply the fixed window strategy with the window size of 120 (this number

refers to the number of log messages) to each set of log data with the same *node_id*. As shown in the Table 3, the average length of a log sequence in this dataset is 55, which is smaller than the window size – 120. This is because the number of log messages with a certain *node_id* can be smaller than 120 and our fixed window strategy is applied to the log messages with the same *node_id*. With this strategy, a log sequence is regarded as an anomalous one if it contains at least one anomalous log message.

Spirit was produced by the Spirit supercomputer at Sandia National Labs [62] with a time span of 2.5 years. There are 272,298,969 log messages in total, including 181,642,697 anomalous messages. Similar to the existing work [36], we sampled the first 12 million log messages (including 360,000 anomalous messages) in the whole dataset as the studied dataset in our work since using the whole dataset requires unaffordable computing and storage resources [36]. We also used the strategy of fixed windows with the window size of 120 to extract log sequences from log messages.

NC was produced by a real-world large-scale online service system in the Network Center of our university. It contains several kinds of anomalies, such as system failures, network issues, and security breach. In total, there are 334,567 log messages, among which 15,394 are anomalous. The annotation process is semi-automatic, where system monitors first report alerts according to the pre-defined rules and operators then manually check them to filter out false alarms. Based on the suggestions from its operators, we adopted the fixed time window strategy with the window size of 5 minutes to extract log sequences following the practical scenario of it. That is, the log messages within a 5-minutes period are grouped as a log sequence.

MC was produced by a real-world large-scale online service system in one influential motor corporation throughout the world, which focuses on developing, simulating, and testing various autonomous driving algorithms. There are 6,148,033 log messages in *MC*, among which 4,833 are labeled as anomalous ones by domain experts following the similar annotation process as *NC*. Similarly, we used the strategy of fixed time windows with the window size of 5 minutes to extract log sequences as well, confirmed by the operators of this system. Due to the confidential policy of the corporation, we have to omit the anomaly ratio of this dataset.

For each dataset, we split it into a training set, a validation set, and a test set in chronological order of log sequences with the ratio of 6:1:3. Here, we did not adopt the widely-used splitting method in the existing work [17, 84], which shuffles all the log sequences before splitting. This is because it could lead to data leakage [66] and thus evade the problem of unseen log events. Our splitting method can ensure all the log sequences in the training set are produced before the log sequences in the test set, which is much closer to the practical scenario and indeed comes across the problem of unseen log events (especially for *BGL* and *Spirit* due to their longer time spans for log data).

4.3 Metrics

In our study, we measured both effectiveness and efficiency of each studied log-based anomaly detection technique.

Since log-based anomaly detection is actually a binary classification problem (i.e., distinguishing anomalies and normal cases), we adopted *Precision* (*Prec.*), *Recall*, and *F1-score* (F_1) as the metrics to measure the effectiveness of each technique following the existing work [17, 29, 47, 79, 84]. *Precision* is computed by $\frac{TP}{TP+FP}$ while *Recall* is computed by $\frac{TP}{TP+FN}$, where *TP*, *FP*, and *FN* refer to the number of true positives (an *anomalous* log sequence is indeed predicted to be *anomalous*), false positives (a *normal* log sequence is predicted to be *anomalous*), and false negatives (an *anomalous* log sequence is predicted to be *normal*), respectively. *F1-score* considers both *Precision* and *Recall*, which is computed by $\frac{2 \cdot (Precision \cdot Recall)}{Precision + Recall}$.

Furthermore, following the existing work [21, 79, 88], we adopted the time spent on building an anomaly detection model (called training time) and the time spent on online anomaly detection (called prediction time) as the efficiency metrics in our study. Although building an anomaly detection model is an offline task, the time spent on it is still important since the anomaly detection model has to be frequently updated due to the frequent software evolution. Please note that training time does not include the time spent on hyper-parameter tuning through grid search, but refers to the time spent on build the anomaly detection model based on the best hyper-parameters after tuning. It is reasonable to exclude the time spent on hyper-parameter tuning through grid search since each technique employs grid search for hyper-parameter tuning. Prediction time refers to the average time spent on predicting an incoming log sequence across the entire test set for each dataset. The time spent on predicting incoming log sequences for online anomaly detection is also quite important since an anomaly detection model is required to monitor the software status in *real time*.

It is also important to measure the average training time for one epoch (called one-epoch training time) for DL-based techniques since their efficiency could be improved after the initial training by training for a few epochs or incremental updates in practice. Moreover, based on one-epoch training time, we can also conclude the efficiency superiority of *SemPCA* over DL-based techniques for the process of grid search.

4.4 Implementations and Configurations

In total, we studied seven log-based anomaly detection techniques. For PLELog, we directly used the released implementation by the existing work [79]. For PCA and LogCluster, we adopted the implementations provided by the existing study [29]. Regarding DeepLog [17], LogAnomaly [47], and LogRobust [84], their implementations are not available and thus we re-implemented them according to the descriptions in their papers. To ensure that our re-implementations are correct and our usage for the released implementations are correct, we conducted a small experiment on HDFS to reproduce the result of each technique according to their original experimental settings. Indeed, for each technique, the difference between our result and the result provided by the original work is minor, i.e., the average F1-score difference is just 0.013 across all the six techniques. Regarding *SemPCA*, we implemented it in Python 3.8 based on scikit-learn 0.24 [1].

Following the practice in deep learning [58, 72], *we determined the hyper-parameter settings of each studied technique through extensive grid search on the corresponding validation set*, in order to make each studied technique achieve the best effectiveness (i.e., F1-score) on each dataset. Please note that the grid search process is not part of *SemPCA* or any other studied techniques, but it is common practice in deep learning [73, 86]. Hence, we employed it for all the studied techniques on each validation set for sufficient and fair comparison. In particular, we developed a toolbox that integrates the implementations of all the seven studied techniques in order to promote future research and practical use. In this toolbox, each technique can be invoked by simply specifying the corresponding configuration. Our toolbox and all the experimental data (also including the parameter settings of each technique used in our study after grid search) can be found at our project homepage: <https://github.com/YangLin-George/SemPCA>. We conducted all the experiments on a server with Ubuntu 18.04 LTS, Intel Xeon Gold 6240C CPU, 128GB RAM, and a NVIDIA RTX3090.

4.5 Experimental Setup

RQ1 aims to investigate whether optimizing the traditional PCA technique through lightweight adaptation can achieve comparable effectiveness with the widely-studied techniques (especially the state-of-the-art DL-based techniques). To answer RQ1, we applied each technique to each

Table 4. Effectiveness comparison among the studied techniques

Technique	<u>HDFS</u>			<u>BGL</u>			<u>Spirit</u>			<u>MC</u>			<u>NC</u>		
	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁
PCA	0.996	0.815	0.897	0.565	1.000	0.722	0.924	0.945	0.934	0.931	0.979	0.954	0.966	0.821	0.887
LogCluster	0.997	0.900	0.946	0.988	0.623	0.766	0.994	0.942	0.968	0.991	0.879	0.932	0.999	0.671	0.802
DeepLog	0.864	0.958	0.909	0.162	0.868	0.273	0.889	0.995	0.939	0.946	0.999	0.972	0.996	0.994	0.995
LogAnomaly	0.933	0.992	0.962	0.151	0.791	0.253	0.883	0.975	0.927	0.945	0.999	0.971	0.979	0.997	0.988
PLELog	0.989	0.957	0.973	0.978	0.998	0.988	0.963	0.980	0.971	0.985	0.994	0.984	1.000	0.999	0.999
LogRobust	0.999	0.997	0.998	0.999	0.999	0.999	0.994	0.892	0.938	0.988	0.982	0.985	0.999	0.994	0.995
SemPCA	0.963	1.000	0.981	0.897	1.000	0.946	0.981	0.920	0.950	0.932	1.000	0.965	0.997	0.912	0.953

dataset, and then measured *Precision*, *Recall*, and *F1-score* to compare these techniques in terms of effectiveness.

Then, we investigated how these studied techniques suffer from the two above-mentioned limitations. Specifically, RQ2 aims to investigate whether the optimized version of the traditional PCA technique can perform more stably under insufficient training data than the existing techniques. To answer RQ2, for each dataset, we constructed a series of training sets by randomly sampling 1%, 2%, 5%, 10%, and 20% of log sequences from the whole training data respectively, which aims to simulate the training sets with different degrees of quality by changing the amounts of training data. Here, we constructed the training sets with much less training data, which aims to simulate the practical scenario suffering from the data hungry problem [21]. Specifically, collecting sufficient training data (especially labeled data) is challenging and costly in practice, and thus the small amount of data is a common factor affecting the quality of training data. If a technique can perform well with various small amounts of training data, it means that this is a stable technique without heavy dependency on training data. Besides, for each studied amount, we repeated the sampling process 10 times for constructing 10 different training sets with the same amount of training data. On the one hand, it can reduce the threat from randomness; On the other hand, the training sets with the same amount of log data could also have different degrees of quality (e.g., different degrees of data imbalance), and thus such a way can help investigate the stability of each technique when controlling for the amount of training data.

Please note that for each dataset, all the log sequences in these constructed training sets are sampled from the whole training set and meanwhile we used the same test set for fair evaluation. With these constructed training sets, we applied each technique to each of them and then measured its effectiveness on the test set of the corresponding dataset.

Subsequently, RQ3 aims to investigate the efficiency of these studied techniques. To answer RQ3, we recorded the training time and prediction time for each technique on each dataset.

5 RESULTS AND ANALYSIS

5.1 RQ1: Effectiveness Comparison

Table 4 shows the comparison results of the seven studied techniques in terms of anomaly detection effectiveness, where the best effectiveness on each dataset is marked as bold. By comparing *SemPCA* with PCA, we found that the former performs significantly better than the latter, e.g., the average improvement in terms of F1-score is 0.101 across all the datasets. **The results demonstrate the significant contribution of the semantic-based log representation method in *SemPCA*.** Moreover, *SemPCA* also outperforms another traditional technique (i.e., LogCluster) on all the datasets (except *Spirit*). For example, the improvements in terms of F1-score on *HDFS*, *BGL*, *MC*, and *NC* range from 3.541% to 23.822%.

Regarding the studied DL-based techniques, the supervised technique (i.e., LogRobust) and the state-of-the-art semi-supervised technique (i.e., PLELog) perform better than the other two (i.e., DeepLog and LogAnomaly). This is as expected since both LogRobust and PLELog carefully handle the problem of unseen log events by designing the semantic-based log representation methods and meanwhile LogRobust incorporates much more information (i.e., the labels of both normal and anomalous log sequences). Surprisingly, the optimized version of the *unsupervised* PCA technique (i.e., *SemPCA*) achieves comparable effectiveness with both LogRobust and PLELog, even though *SemPCA* does not require any label information and relies on a traditional algorithm (i.e., PCA) for building an anomaly detection model. The F1-score differences between *SemPCA* and PLELog only range from 0.021 to 0.046 for *BGL*, *Spirit*, *MC*, and *NC*, and those between *SemPCA* and LogRobust only range from 0.017 to 0.053 for *HDFS*, *BGL*, *MC*, and *NC*. Even *SemPCA* performs slightly better than LogRobust on *Spirit* and PLELog on *HDFS* in terms of F1-score. The reason why LogRobust does not perform very well on *Spirit* may be that LogRobust is supervised, i.e., learning patterns from both normal and abnormal log sequences, but the imbalance problem on *Spirit* is very significant. The results demonstrate the great potential of *SemPCA* in log-based anomaly detection, indicating that properly optimizing traditional techniques is also a promising direction in this area.

From Table 4, we also found that on *BGL*, both DeepLog and LogAnomaly perform significantly worse than all the other techniques (even the traditional ones, i.e., PCA and LogCluster). This is because the *BGL* dataset suffers from the most serious unseen-log-event problem among all the studied datasets. Both DeepLog and LogAnomaly predict several possible log events that can follow the subsequence of log events in a log sequence. In this way, when coming across an unseen log event that is dissimilar to the predicted ones according to the log event vectors, the log sequence is directly identified as an anomaly. Although LogAnomaly also extracts semantics of log events through synonyms-antonyms-based log representation, the way of predicting an anomaly makes it hard to handle the problem of unseen log events. On the other hand, although PCA and LogCluster do not handle this problem, they directly ignore unseen log events and count the occurrence of seen log events for anomaly detection, which is less aggressive to treat the log sequences with unseen log events than DeepLog and LogAnomaly. Indeed, all these techniques without well relieving the problem of unseen log events perform much worse than those incorporating semantic-based log representation on *BGL*, which further confirms the necessity of relieving this problem in log-based anomaly detection.

Finding 3: *SemPCA* significantly outperforms PCA by incorporating lightweight semantic-based log representation, and achieves comparable effectiveness with the supervised DL-based technique (i.e., LogRobust) and the semi-supervised DL-based technique (i.e., PLELog).

5.2 RQ2: Stability Comparison

Table 5 shows the effectiveness of each studied technique under different amounts of training data. For each technique on each dataset under each studied amount of training data, we calculated the average F1-score (avg. denoted in the table) across 10 training sets with the same amount of training data, and calculated the corresponding standard deviation (σ denoted in the table). Here, we mainly studied small amounts of training data in order to simulate the common data-hungry problem in the practical use of deep learning [21].

From Table 5, in general, the studied DL-based techniques perform much worse under small amounts of training data than using the whole training data (shown in Table 4) in terms of average F1-score. This is as expected since DL-based techniques tend to rely on a huge amount

Table 5. Average and standard deviation of F1-score of each studied technique under different amounts of training data.

Dataset	Ratio	PCA		LogCluster		DeepLog		LogAnomaly		PLELog		LogRobust		SemPCA	
		avg.	σ	avg.	σ	avg.	σ	avg.	σ	avg.	σ	avg.	σ	avg.	σ
HDFS	1	0.872	0.022	0.946	0.001	0.853	0.043	0.932	0.015	0.767	0.363	0.924	0.036	0.980	0.001
	2	0.881	0.023	0.947	0.001	0.822	0.074	0.944	0.014	0.979	0.009	0.963	0.019	0.978	0.007
	5	0.884	0.019	0.947	0.000	0.784	0.028	0.795	0.057	0.785	0.083	0.981	0.011	0.981	0.000
	10	0.888	0.017	0.947	0.001	0.875	0.060	0.794	0.032	0.727	0.046	0.989	0.006	0.976	0.009
	20	0.888	0.016	0.947	0.001	0.893	0.045	0.843	0.051	0.638	0.201	0.997	0.001	0.981	0.000
BGL	1	0.709	0.007	0.670	0.029	0.267	0.017	0.256	0.010	0.755	0.061	0.718	0.163	0.908	0.038
	2	0.714	0.013	0.695	0.029	0.249	0.013	0.237	0.015	0.667	0.186	0.911	0.038	0.928	0.033
	5	0.714	0.014	0.741	0.017	0.244	0.015	0.227	0.001	0.870	0.076	0.914	0.105	0.941	0.004
	10	0.719	0.010	0.757	0.008	0.241	0.017	0.227	0.000	0.961	0.030	0.963	0.027	0.937	0.006
	20	0.717	0.010	0.762	0.005	0.236	0.016	0.227	0.000	0.961	0.028	0.972	0.027	0.945	0.005
Spirit	1	0.073	0.004	0.904	0.035	0.732	0.028	0.918	0.010	0.809	0.101	0.227	0.321	0.916	0.036
	2	0.205	0.266	0.924	0.015	0.708	0.067	0.938	0.012	0.700	0.093	0.313	0.330	0.919	0.032
	5	0.325	0.311	0.947	0.019	0.663	0.084	0.957	0.018	0.836	0.102	0.294	0.311	0.923	0.030
	10	0.595	0.359	0.955	0.016	0.614	0.087	0.931	0.082	0.857	0.083	0.363	0.338	0.933	0.002
	20	0.894	0.080	0.963	0.011	0.588	0.060	0.951	0.033	0.853	0.053	0.606	0.296	0.924	0.030
MC	1	0.943	0.011	0.947	0.014	0.989	0.001	0.856	0.007	0.963	0.040	0.788	0.306	0.964	0.001
	2	0.941	0.016	0.942	0.011	0.989	0.001	0.867	0.005	0.979	0.038	0.947	0.033	0.963	0.001
	5	0.944	0.018	0.937	0.011	0.989	0.001	0.874	0.002	0.922	0.039	0.976	0.036	0.964	0.001
	10	0.952	0.014	0.936	0.005	0.989	0.001	0.875	0.001	0.890	0.047	0.952	0.057	0.964	0.001
	20	0.955	0.010	0.932	0.004	0.989	0.001	0.876	0.001	0.912	0.029	0.911	0.052	0.964	0.000
NC	1	0.882	0.000	0.789	0.000	0.970	0.008	0.978	0.003	0.963	0.042	0.979	0.012	0.948	0.033
	2	0.882	0.000	0.789	0.000	0.972	0.009	0.980	0.004	0.979	0.040	0.990	0.006	0.935	0.030
	5	0.883	0.002	0.789	0.000	0.968	0.006	0.983	0.002	0.922	0.041	0.994	0.002	0.980	0.004
	10	0.884	0.003	0.789	0.000	0.980	0.009	0.989	0.001	0.890	0.049	0.997	0.001	0.980	0.004
	20	0.885	0.003	0.789	0.000	0.978	0.009	0.989	0.001	0.912	0.031	0.998	0.001	0.980	0.004

of training data to learn the parameters in complex neural networks [55, 60]. In contrast, the traditional techniques (i.e., *SemPCA*, PCA, LogCluster) perform stably regardless of the studied amounts of training data. In particular, when using 1% of training data, the optimized version of the unsupervised PCA technique (i.e., *SemPCA*) performs the best among these studied techniques in most cases. For example, the unsupervised *SemPCA* performs much better than the supervised LogRobust in four (out of five) datasets under 1% of training data, i.e., the improvements of *SemPCA* over LogRobust in terms of average F1-score are 6.061% on *HDFS*, 26.462% on *BGL*, 136.082% *Spirit*, and 22.335% on *MC*. The results demonstrate that *SemPCA* is more stable and less dependent on the amount of training data than the advanced DL-based techniques.

Please note that even though 1% is a very small ratio, 1% of training data does not mean the very small amount of training data. For example, for *HDFS*, 1% of training data contains 57,506 log sequences. In practice, labeling such a large number of training data is still very expensive, indicating that the data hungry problem could be more serious in practice and thus the effectiveness of these DL-based techniques could be affected more deeply.

From the results of standard deviation in Table 5, *SemPCA* (as well as the other two traditional techniques) perform more stably than the studied DL-based techniques. For example, on *Spirit*, the standard deviation of *SemPCA* ranges from 0.002 to 0.036 while those of the supervised LogRobust

and the state-of-the-art semi-supervised PLELog range from 0.280 to 0.335 and from 0.053 to 0.102 respectively, across all the studied amounts of training data. The mean of standard deviation for *SemPCA* is 0.012 across all the cases, which is 88.000%, 84.211%, 57.143%, and 33.333% smaller than that of LogRobust, PLELog, DeepLog, and LogAnomaly respectively. The results demonstrate that the optimized version of the traditional PCA technique (i.e., *SemPCA*) is much more stable than the studied DL-based techniques when controlling for the amount of training data, indicating that these studied DL-based techniques also more heavily depend on the quality of training data. In practice, collecting and labeling sufficient *high-quality* training data is quite challenging, which further hinders the practical use of these DL-based techniques.

In particular, LogRobust performs significantly worse and less stably than all the other techniques on *Spirit* under small amounts of training data. The main reason lies in that LogRobust requires to learn patterns from both normal and abnormal log sequences and meanwhile the distribution of the data in *Spirit* is more diverse, and thus it requires much more training data for building an accurate anomaly detection model. Indeed, when using the whole training data, the effectiveness of LogRobust is significantly improved. However, all the other techniques either just learn patterns from normal log sequences or detect anomalies via outlier analysis, and thus the dependency on the training data amount is relatively small. The results further confirm the challenge of applying the supervised log-based anomaly detection techniques to the practice.

Finding 4: The optimized version of the traditional PCA technique (i.e., *SemPCA*) is more stable and less dependent on the amount and quality of training data than the advanced DL-based techniques. When using small amounts of training data, the former even performs better than the latter in terms of average F1-score.

5.3 RQ3: Efficiency Comparison

Table 6 shows the efficiency comparison results among these studied techniques. For each technique, we reported the training time (*train*, in seconds), the prediction time (*pred*, in milliseconds), the one-epoch training time (*one-epoch*, in seconds), on each dataset.

From this table, the traditional techniques spend significantly shorter training time than the advanced DL-based techniques on each dataset. For example, the average training time of *SemPCA* is only 0.279 seconds while that of DeepLog, LogAnomaly, PLELog, and LogRobust are 1,332.405, 58,674.582, 1,723.260, and 1,620.668 seconds, respectively. Among the four DL-based techniques, LogAnomaly was more than 10 times slower than the others, especially on the *Spirit* dataset. This is because LogAnomaly predicts the next log event based on the previous ones, causing that each log sequence can be transformed to multiple instances and thus the volume of training data can be significantly increased. Moreover, the inclusion of two LSTMs (one incorporating quantity features and another incorporating semantic features) further aggravates the training cost. Moreover, the training time for *SemPCA* is stable for different datasets with different scales of training data (ranging from 0.017 seconds to 1.053 seconds), while that for DL-based techniques is affected significantly (ranging from 29.847 seconds to 2,566.723 seconds for DeepLog, ranging from 1,862.281 seconds to 148,004.419 seconds for LogAnomaly, ranging from 57.967 seconds to 3,252.943 seconds for LogRobust, and ranging from 56.747 seconds to 3,782.365 seconds for PLELog). The results further demonstrate the superiority of *SemPCA* in terms of efficiency across different datasets.

Furthermore, we found that the time spent on the whole process of building an anomaly detection with *SemPCA* is still significantly shorter than the one-epoch training time for each DL-based technique. For example, the whole training time for *SemPCA* is 0.279 seconds, while the one-epoch

Table 6. Efficiency comparison among the studied techniques.

Dataset		PCA	LogCluster	DeepLog	LogAnomaly	LogRobust	PLELog	SemPCA
HDFS	<i>train</i>	0.103	10.195	1,782.453	72,933.082	3,252.943	3,782.365	1.053
	<i>one-epoch</i>	-	-	44.562	1,823.327	162.602	189.118	-
	<i>pred.</i>	0.002	0.046	0.042	0.048	0.200	0.197	0.003
BGL	<i>train</i>	0.118	6.859	645.082	36,594.352	1,016.560	998.542	0.101
	<i>one-epoch</i>	-	-	16.127	3,148.195	50.828	49.925	-
	<i>pred.</i>	0.026	0.437	0.384	0.282	0.367	0.268	0.028
Spirit	<i>train</i>	0.983	48.936	2,566.723	>24 hours	2,361.429	2,855.766	0.164
	<i>one-epoch</i>	-	-	64.168	36,821.91	118.071	142.788	-
	<i>pred.</i>	0.303	0.611	19.326	152.793	1.704	1.251	0.016
MC	<i>train</i>	0.011	0.075	29.847	1,862.281	57.967	56.747	0.017
	<i>one-epoch</i>	-	-	0.746	46.557	2.898	2.837	-
	<i>pred.</i>	0.014	0.018	1.131	1.222	0.152	0.114	0.003
NC	<i>train</i>	0.017	2.955	1,637.920	33,975.866	1,414.442	922.881	0.061
	<i>one-epoch</i>	-	-	40.948	849.375	70.722	46.144	-
	<i>pred.</i>	0.068	0.017	0.798	0.194	0.871	0.759	0.015
Avg.	<i>train</i>	0.246	13.804	1,332.405	58,674.582	1,620.668	1,723.260	0.279
	<i>one-epoch</i>	-	-	33.310	8,537.873	81.024	86.162	-
	<i>pred.</i>	0.0826	0.226	4.336	30.908	0.659	0.518	0.013

training time for LogRobust is 81.024 seconds, on average across all the datasets. That is, even though the efficiency of DL-based techniques could be improved after the initial training by training for a few epochs or incremental updates, *SemPCA* is still much more efficient than DL-based techniques. Based on these results, we can also infer that the grid-search process for DL-based techniques can be more time-consuming than that for *SemPCA* (since an anomaly detection model has to be built and evaluated under each explored hyper-parameter setting in the grid search process), which further indicates the efficiency of *SemPCA*.

In terms of average prediction time, that of all the studied techniques is acceptable, and *SemPCA* and PCA spend shorter prediction time than the other techniques.

To sum up, *SemPCA* and PCA are much more efficient than the advanced DL-based techniques in terms of both training and prediction time. Such high efficiency is helpful to update the anomaly detection model in time, tune the hyper-parameters sufficiently and efficiently, and monitor the system in real time. The results demonstrate the practicability of *SemPCA*, especially when comprehensively considering the effectiveness, efficiency, and stability.

Finding 5: All the studied techniques have acceptable prediction time and that of *SemPCA* and PCA is shorter. However, the traditional techniques (i.e., *SemPCA*, PCA, and LogCluster) spend significantly shorter training time than the advanced DL-based techniques. This also confirms that incorporating lightweight semantic-based log representation into the PCA technique does not damage its advantage of high efficiency.

Summary: The unsupervised *SemPCA* (i.e., the optimized version of the traditional PCA technique through simple adaptation) achieves comparable effectiveness with the supervised and

semi-supervised DL-based techniques while is more stable, less dependent on training data, and more efficient. That demonstrates the great potential of *SemPCA* for more practical log-based anomaly detection, and confirms that such a traditional technique can still excel after simple but useful adaptation.

Implication: Please note that the point of our study is not to deprecate deep learning in the area of log-based anomaly detection, but evaluate the existing DL-based techniques from a novel perspective. That is, we investigated whether the existing DL-based techniques have prominent advantage compared with the traditional techniques through simple but useful adaptation in terms of comprehensive metrics (i.e., effectiveness, efficiency, and stability). Although our conclusions are relatively negative for the existing DL-based techniques in terms of the studied metrics, it is still exciting to see new DL-based techniques that can significantly outperform all the existing techniques (including the traditional techniques with some optimizations) in terms of effectiveness but cannot further damage the efficiency and stability under insufficient training data. In the future, when evaluating newly-proposed DL-based techniques, optimizing the traditional techniques accordingly for comparison should be also noticed in order to confirm the contribution of advanced DL models to the significant effectiveness improvement.

6 DISCUSSION

6.1 Why does *SemPCA* Work?

Based on our study, the traditional PCA algorithm can still excel by incorporating a lightweight semantic-based log representation in log-based anomaly detection. The success of *SemPCA* is inspiring, and we further analyzed the insight behind its success. The main reasons are threefold.

First, incorporating semantic-based log representation is helpful to relieve the problem of unseen log events. The original PCA represents a log sequence as a event count vector by counting the occurrence of each log event (seen in training data) in the log sequence, which directly ignores the unseen log events in incoming log sequences and thus negatively affects its effectiveness. However, the semantic-based log representation method in *SemPCA* can map the unseen log events in incoming log sequences to the log events seen in training data by measuring their semantic similarity.

Second, the semantic-based log representation method in *SemPCA* can facilitate the understanding of semantic differences between anomalous and normal log sequences, and thus *SemPCA* is effective to detect more obscure anomalies. However, the event count vector method in the original PCA is just effective to detect the anomalies exhibited by the exceptional number of log events, which are usually obvious and could be also captured by the semantic differences.

Third, the number of anomalies tends to be much smaller than that of normal cases in practice [17, 47, 79], leading to the problem of imbalanced data. Such imbalanced data could negatively affect the effectiveness of supervised DL models, which has been widely recognized in the area of deep learning [4, 16], while it has little influence on the unsupervised PCA algorithm since PCA can effectively learn patterns from the majority class (i.e., normal log sequences) and then determine whether an incoming log sequence is an outlier with respect to the learned patterns. Therefore, in our study, *SemPCA* performs more stably under a variety of training sets.

In addition, the adopted lightweight semantic-based log representation method in *SemPCA* does not consider the order information of log events, and thus the optimized version of the traditional PCA technique cannot detect the anomalies that are only reflected by log-event orders. In fact, the existing study analyzed tens of thousands of anomalies related to logs in industry and found that (1) such a kind of anomalies are not common in practice; (2) the anomalies involving abnormal log-event orders tend to also involve abnormal log-event counts or log-event types [88]. Hence,

Table 7. Effectiveness comparison between SVM/LogCluster and *SemSVM*/*SemLogCluster* on all the five used datasets

Dataset	SVM			<i>SemSVM</i>			<i>LogCluster</i>			<i>SemLogCluster</i>		
	<i>Prec.</i>	<i>Recall</i>	<i>F₁</i>	<i>Prec.</i>	<i>Recall</i>	<i>F₁</i>	<i>Prec.</i>	<i>Recall</i>	<i>F₁</i>	<i>Prec.</i>	<i>Recall</i>	<i>F₁</i>
<i>HDFS</i>	0.999	0.662	0.796	0.999	0.974	0.986	0.997	0.900	0.946	0.963	0.985	0.972
<i>BGL</i>	0.799	0.853	0.825	0.998	0.889	0.940	0.988	0.623	0.766	0.972	0.654	0.782
<i>Spirit</i>	0.993	0.959	0.976	0.981	0.985	0.983	0.994	0.942	0.968	0.995	0.936	0.965
<i>MC</i>	0.933	1.000	0.965	0.932	0.999	0.964	0.991	0.879	0.932	0.952	0.986	0.968
<i>NC</i>	1.000	0.643	0.783	1.000	0.823	0.903	0.999	0.671	0.802	0.996	0.925	0.959
Average	0.944	0.823	0.869	0.982	0.934	0.955	0.993	0.803	0.883	0.976	0.897	0.929

many of such a kind of anomalies can be also detected by *SemPCA*. In the future, we can further optimize traditional techniques by incorporate the order information of log events.

6.2 Can Other Traditional Techniques Excel through Semantic-based Log Representation?

In the study, we took PCA as the representative to investigate whether traditional techniques through some optimizations can achieve comparable effectiveness with advanced DL-based techniques in log-based anomaly detection. Indeed, we found that incorporating semantic-based log representation is helpful to largely improve the effectiveness of PCA almost without damaging its efficiency, since it can relieve the problem of unseen log events well. Here, we further investigated whether the conclusion can be generalized to other traditional techniques. Specifically, we conducted an experiment on the other two traditional techniques, i.e., LogCluster [40] and SVM [85]. The former is semi-supervised that has been introduced in Section 2.2.1, while the latter is supervised that first uses the event count vector method for log representation like PCA and LogCluster and then builds a classifier via the SVM algorithm for anomaly detection. Based on them, we constructed *SemLogCluster* and *SemSVM* by incorporating the lightweight semantic-based log representation method used in *SemPCA*, respectively.

Table 7 shows the comparison results between SVM/LogCluster and *SemSVM*/*SemLogCluster* on all the five used datasets. We found that after incorporating semantic-based log representation in both LogCluster and SVM, their effectiveness can be effectively improved. Specifically, the average improvement of *SemLogCluster* over LogCluster is 5.210% and that of *SemSVM* over SVM is 9.896%, in terms of F1-score, which makes their effectiveness close to the supervised LogRobust and the semi-supervised PLELog except *SemLogCluster* on the *BGL* dataset. The results demonstrate the generalizability of the conclusions from our study to a large extent and further confirm the great potential of traditional techniques for more practical log-based anomaly detection.

We manually analyzed the reason about the relatively small improvement of *SemLogCluster* over LogCluster on the *BGL* dataset. The reason lies in that we directly used the pre-trained word vectors on the Common Crawl Corpus dataset using the Fast-Text toolkit for semantic representation following the existing work [79, 84], and 8.922% of tokens in the *BGL* dataset are Out-Of-Vocabulary (OOV) with regard to the pre-trained word vectors. Further, the OOV tokens are uniformly represented by a specialized token (such as “<ooov>”), and thus the distance between log sequences measured by the clustering-based technique can be very small based on the semantic representation on the *BGL* dataset, which increases the difficulty of precisely clustering them. Nevertheless, *SemLogCluster* still outperforms LogCluster, demonstrating the usefulness of semantic-based log representation for traditional anomaly detection techniques to some degree.

#	Event	Log Message	#	Event	Log Message	
1	11	773760 double-hammer alignment exceptions	1	11	773760 double-hammer alignment exceptions	11:21
2	254	iar 003a9260 dear 00e27cf8	2	254	iar 003a9260 dear 00e27cf8	
3	11	126817526 double-hammer alignment exceptions	3	11	134976970 double-hammer alignment exceptions	
4	11	137814690 double-hammer alignment exceptions	4	11	148476854 double-hammer alignment exceptions	
5	423	0 microseconds spent in the rbs signal handler during 0 calls.	5	53	critical input interrupt (unit=0x0b bit=0x03)	
6	424	10684 total interrupts. 0 critical input interrupts.	6	120	suppressing further interrupts of same type	
7	423	0 microseconds spent in the rbs signal handler during 0 calls.	7	423	0 microseconds spent in the rbs signal handler during 0 calls.	
8	424	10684 total interrupts. 0 critical input interrupts.	8	424	10736 total interrupts. 0 critical input interrupts.	
9	324	Error receiving packet on tree network	9	423	0 microseconds spent in the rbs signal handler during 0 calls.	
			10	424	10736 total interrupts. 0 critical input interrupts.	

(a) An anomalous log sequence
(b) A normal log sequence

Fig. 4. An anomalous log sequence extracted from *BGL*. Line 9 of log sequence (a) is labeled by experts as anomalous log message.

In the future, we can improve the effectiveness of *SemLogCluster* by incorporating more effective semantic-based log representation methods.

Here, we did not study the IM technique (Invariants Mining [42]). IM mines the invariants (or patterns) existing in log sequences (such as “open file” and “close file”) based on the occurrence frequency of log events and detects an anomaly by determining whether a log sequence violates the invariants. However, after representing log sequences as the form of semantic vectors, it is hard to mine such invariants due to the incompatibility with the principle underlying IM, and thus it cannot be used for anomaly detection.

6.3 Case Study

We further illustrate why *SemPCA* works well through two examples in this section.

6.3.1 Case 1. Figure 4 shows two similar log sequences extracted from the test set of *BGL*. The log sequence (a) is anomalous, while the log sequence (b) is normal. In our experiment, PCA failed to identify the anomalous log sequence (a) as an anomaly, while *SemPCA* succeeded. The reason is that the log events 423 and 424 are unseen in the training set, and thus PCA cannot count them in the event count vector. Hence, the event count vector for this log sequence is: [11 : 3, 254 : 1, 324 : 1]. After mapping this vector to the abnormal space, the SPE value of the projection is 17.391, which is smaller than the anomaly detection threshold, and thus PCA mis-reported the anomalous log sequence (a) as a normal one. In contrast, *SemPCA* can capture the semantics of the unseen log events 423 and 424 through semantic-based log representation, which help identify this anomaly, demonstrating the contribution of semantic-based log representation for unseen log events.

Besides, DeepLog is also affected by unseen log events. In this example, DeepLog mis-reported the normal log sequence (b) as an anomaly. As presented in Section 2, DeepLog is designed to predict the next log event according to the log events before it. When predicting the log event at Line 10, DeepLog takes the first nine lines of log events as input. Since the log events 423 and 424 are unseen in training data, DeepLog neither represents both of them nor predicts the log event 424, thereby identifying it as an anomaly. That demonstrates the necessity of relieving the problem of unseen log events.

6.3.2 Case 2. Figure 5 is a part of an anomalous log sequence extracted from the test set of *BGL*. In this log sequence, the log message at Line 13 is labeled as anomalous by experts. When we used 20% of training data to build an anomaly detection model through LogRobust in the experiment presented in Section 5.2, LogRobust did not identify this anomaly, while LogRobust succeed when using the entire training data to build an anomaly detection model. The reason for the different results lies in the different distribution of this log event in corresponding training data. Specifically, the frequency of occurrence of this log message is significantly low in the sampled 20% of training data, compared to that in the entire training data, and thus LogRobust cannot effectively learn the anomalous information in it based on the insufficient training data. That demonstrates that LogRobust heavily relies on the quality of training data.

Line	Event	Log Message
1	3	generating core.10092
...	...	...
7	144	floating point alignment exceptions
...	...	...
12	3	generating core.10092
13	248	kernel terminated for reason 1002
14	250	bad message header: packet index 131070 greater than max 366
15	423	0 microseconds spent in the rbs signal handler during 0 calls.
16	424	10678 total interrupts. 0 critical input interrupts.
...	...	...

Fig. 5. An anomalous log sequence extracted from *BGL*. The line 13 is labeled by experts as anomalous log message.

In contrast, *SemPCA* can identify this log sequence as anomalous regardless of using 20% of training data or the entire training data in our experiment. This is because the insight of *SemPCA* is to detect outliers as anomalies. That is, as long as the semantics of anomalous log sequences through semantic-based log representation is minority, *SemPCA* can stably identify these anomalies based on the SPE values in the abnormal space. This example matches the insight of *SemPCA* for both sets of training data in the experiment. That is, even though the frequency of occurrence of this log message is relatively high in the entire training data, the semantics of this log sequence is still minority relative to all log sequences.

6.4 Threats to Validity

The threat to *internal* validity mainly lies in our implementation. In our study, we implemented *SemPCA* and re-implemented DeepLog, LogRobust, and LogAnomaly. To reduce this kind of threat, we used some mature libraries such as PyTorch and three authors have carefully checked all the code. In particular, as presented in Section 4.4, we also conducted a replication study to confirm the correctness of our re-implementations. All our code has been released for future replication.

The threat to *external* validity mainly lies in the used datasets. We used five datasets in our study, including three widely-used public datasets (i.e., *HDFS*, *BGL*, *Spirit*) and two industrial datasets (i.e., *MC* and *NC*), which may not represent other datasets. Actually, our used datasets are diverse by collecting from different systems in different organizations, which can help reduce this kind of threat to some degree. In the future, we will repeat our experiments on more datasets with greater diversity, especially for the datasets with more large-scale and high-dimension training data.

The threat to *construct* validity mainly lies in the hyper-parameter setting for each technique. To reduce this kind of threat, for each technique on each dataset, we conducted grid search on the corresponding validation set following the practice in the area of DL [58]. In this way, we can try the best to obtain the optimal hyper-parameter setting for each technique on each dataset, which facilitates the fair comparison among the studied techniques.

Regarding PCA (as well as *SemPCA*), it has a built-in method (i.e., Q-statistics [33]) to automatically determine the setting of the hyper-parameter (i.e., the threshold of the projection length in abnormal space). For fair comparison with other techniques, we still applied grid search to both *SemPCA* and PCA (instead of Q-statistics), and grid search can indeed help find a better hyper-parameter setting than Q-statistics. Here, we added an experiment to compare grid search and Q-statistics on the basis of *SemPCA* to show the necessity of grid search. That is, we evaluated the effectiveness of *SemPCA* under the hyper-parameter setting determined by Q-statistics (named *SemPCA_{auto}* for ease of presentation) for comparison. Table 8 shows the comparison results on

Table 8. Effectiveness comparison among the variants.

Technique	<u>HDFS</u>			<u>BGL</u>			<u>Spirit</u>			<u>MC</u>			<u>NC</u>		
	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁	Prec.	Recall	F ₁
<i>SemPCA_{auto}</i>	0.775	0.458	0.576	0.339	0.456	0.389	0.322	0.991	0.486	1.000	0.862	0.926	0.998	0.846	0.916
<i>SemPCA</i>	0.963	1.000	0.981	0.897	1.000	0.946	0.981	0.920	0.950	0.932	1.000	0.965	0.997	0.912	0.953

each dataset. We found that *SemPCA* largely outperforms *SemPCA_{auto}* in terms of F1-score, demonstrating the superiority of grid search over the built-in Q-statistics method in PCA. In particular, the average time spent on the grid search process by *SemPCA* is just 10.296 seconds across all the datasets, indicating that grid search does not incur too much cost for *SemPCA*.

7 RELATED WORK

Besides the six most widely-studied techniques (also studied in our work) introduced in Section 2.2, there are many other log-based anomaly detection techniques in the literature [20, 22, 27, 42, 43, 69, 74, 78, 82, 83, 89, 90]. For example, regarding traditional techniques, Yamanishi et al. [78] proposed a Hidden Markov Model to model log sequences. Lou et al. [42] proposed to learn invariants in normal log sequences for anomaly detection. All these existing traditional techniques do not consider the problem of unseen log events. Regarding DL-based techniques, they tend to design various DL models for anomaly detection. For example, Lu et al. [43] built a Convolutional Neural Network (CNN) for anomaly detection. Xia et al [74] and Zhao et al. [89] applied Generative Adversarial Network to learn from normal log sequences for anomaly detection. Zhou et al. [90] proposed to use numerical values in log messages for anomaly detection. Zhang et al. [83] proposed a content-aware Transformer to represent log events and applied a Transformer-based decoder to perform anomaly detection. Xie et al. [75] proposed to use the Graph Transformer network to combine both graph structure and log semantics to detect anomalies. Zhang et al. [82] proposed a Graph-based Deep Learning framework to detect anomalies in micro-service system.

Different from them, our work aims to investigate whether the existing traditional techniques through some optimizations can achieve comparable effectiveness with advanced DL-based techniques in log-based anomaly detection. Our results show that by incorporating lightweight semantic-based log representation, the optimized version of the PCA technique (i.e., *SemPCA*) can help achieve more practical log-based anomaly detection. Moreover, the generalizability of our conclusions on other traditional techniques has been investigated in Section 6.2. Please note that *SemPCA* is different from the dimension reduction algorithm (i.e., FastICA) used in PLELog, since the former aims to detect anomalies through outlier detection in the projected abnormal space while the latter aims to speed up the clustering process through dimension reduction.

Since our work is an empirical study on log-based anomaly detection, the existing empirical studies in this area are also relevant [12, 25, 28, 29, 37, 88]. For example, He et al. [29] conducted an empirical study to compare the effectiveness of traditional ML-based log-based anomaly detection techniques. Furthermore, He et al. [28] conducted an empirical study to investigate the real significant challenges of log analysis techniques in the industry (i.e., Microsoft) and summarized several possible Opportunities including increasing the efficiency of log-based anomaly detection. Similarly, Le et al. [37] conducted an empirical study to investigate some DL-based techniques. Chen et al. [12] empirically investigated the effectiveness of state-of-the-art DL-based techniques, and summarized several challenges about their practical use (which can also confirm the necessity of our work). Zhao et al. [88] empirically investigated various types of anomalies in real-world log data. Different from them, our study aims to explore the potential of traditional techniques for more practical anomaly detection following the idea of “try-with-simpler”. In our study, we compared

the optimized version of the PCA technique (i.e., *SemPCA*) with four widely-studied DL-based techniques and two widely-studied traditional techniques in terms of effectiveness, efficiency, and stability.

In addition, the potential of traditional ML algorithms has been evaluated and demonstrated in the tasks of linking questions posted on Stack Overflow [21] and linguistic smell detection [19], which also compare the traditional ML algorithms with state-of-the-art DL-based techniques. Different from them, we are the first to explore and demonstrate the potential of traditional techniques in log-based anomaly detection. Due to different characteristics of different tasks, the adopted optimizations for the studied traditional techniques are completely different. Our work further confirms the idea of “try-with-simpler”.

8 CONCLUSION

In this work, we conducted an extensive study, as the first attempt, to explore the potential of traditional techniques for more practical log-based anomaly detection by optimizing the traditional PCA technique with a lightweight semantic-based log representation method. We call the optimized version of the PCA technique *SemPCA*. Such adaptation on the PCA technique aims to address the problem of unseen log events and thus improve the anomaly detection effectiveness. The results on five datasets show that the unsupervised *SemPCA* achieves comparable effectiveness with the supervised/semi-supervised DL-based techniques while is more efficient, more stable, and less dependent on training data. Our results confirm that the traditional technique can still excel in log-based anomaly detection after simple but useful adaptation, further confirming the idea of “try-with-simpler”.

ACKNOWLEDGMENTS

We gratefully acknowledge the financial support provided by the National Natural Science Foundation of China Grant Nos. 62322208, 62232001, and CCF Young Elite Scientists Sponsorship Program (by CAST) which made this research possible. We would like to express our sincere appreciation to the anonymous reviewers and the editors for their constructive suggestions, which have greatly improved the quality of this paper.

REFERENCES

- [1] Accessed: 2022. Scikit-Learn. <https://scikit-learn.org/stable/index.html>.
- [2] Hervé Abdi and Lynne J Williams. 2010. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics* 2, 4 (2010), 433–459.
- [3] Titus Barik, Robert DeLine, Steven Mark Drucker, and Danyel Fisher. 2016. The bones of the system: a case study of logging and telemetry at Microsoft. In *ICSE (Companion Volume)*. ACM, 92–101.
- [4] Mohammad Mahdi Bejani and Mehdi Ghatte. 2021. A systematic review on overfitting control in shallow and deep neural networks. *Artif. Intell. Rev.* 54, 8 (2021), 6391–6438.
- [5] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomáš Mikolov. 2017. Enriching Word Vectors with Subword Information. *Trans. Assoc. Comput. Linguistics* 5 (2017), 135–146.
- [6] Jakub Breier and Jana Branišová. 2015. Anomaly detection from log files using data mining techniques. In *Information Science and Applications*. Springer, 449–457.
- [7] Christian Callegari, Lisa Donatini, Stefano Giordano, and Michele Pagano. 2018. Improving stability of PCA-based network anomaly detection by means of kernel-PCA. *Int. J. Comput. Sci. Eng.* 16, 1 (2018), 9–16.
- [8] Junjie Chen, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. An empirical investigation of incident triage for online service systems. In *ICSE (SEIP)*. IEEE / ACM, 111–120.
- [9] Junjie Chen, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. Continuous Incident Triage for Large-Scale Online Service Systems. In *ASE*. IEEE, 364–375.
- [10] Mike Y. Chen, Alice X. Zheng, Jim Lloyd, Michael I. Jordan, and Eric A. Brewer. 2004. Failure Diagnosis Using Decision Trees. In *ICAC*. IEEE Computer Society, 36–43.

- [11] Zhichao Chen, Junjie Chen, Weijing Wang, Jianyi Zhou, Meng Wang, Xiang Chen, Shan Zhou, and Jianmin Wang. 2023. Exploring Better Black-Box Test Case Prioritization via Log Analysis. *ACM Trans. Softw. Eng. Methodol.* 32, 3 (2023), 72:1–72:32.
- [12] Zhuangbin Chen, Jinyang Liu, Wenwei Gu, Yuxin Su, and Michael R. Lyu. 2021. Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection. *CoRR* abs/2107.05908 (2021).
- [13] Junyoung Chung, Çağlar Gülçehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. *CoRR* abs/1412.3555 (2014).
- [14] Hetong Dai, Heng Li, Weiyi Shang, Tse-Hsun Chen, and Che-Shao Chen. 2020. Logram: Efficient Log Parsing Using n-Gram Dictionaries. *CoRR* abs/2001.03038 (2020).
- [15] Biplob Debnath, Mohiuddin Solaimani, Muhammad Ali Gulzar, Nipun Arora, Cristian Lumezanu, Jianwu Xu, Bo Zong, Hui Zhang, Guofei Jiang, and Latifur Khan. 2018. LogLens: A Real-Time Log Analysis System. In *ICDCS*. IEEE Computer Society, 1052–1062.
- [16] Claudio Filipi Gonçalves dos Santos and João Paulo Papa. 2022. Avoiding Overfitting: A Survey on Regularization Methods for Convolutional Neural Networks. *CoRR* abs/2201.03299 (2022).
- [17] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *CCS*. ACM, 1285–1298.
- [18] Ricardo Dunia and S Joe Qin. 1997. Multi-dimensional fault diagnosis using a subspace approach. In *American Control Conference*. Citeseer.
- [19] Sarah Fakhoury, Venera Arnaoudova, Cedric Noiseux, Foutse Khomh, and Giuliano Antoniol. 2018. Keep it simple: Is deep learning good for linguistic smell detection?. In *SANER*. IEEE Computer Society, 602–611.
- [20] Markus Fält, Stefan Forsström, and Tingting Zhang. 2021. Machine Learning Based Anomaly Detection of Log Files Using Ensemble Learning and Self-Attention. In *ICSRS*. IEEE, 209–215.
- [21] Wei Fu and Tim Menzies. 2017. Easy over hard: a case study on deep learning. In *ESEC/SIGSOFT FSE*. ACM, 49–60.
- [22] Nico Görnitz, Mikio L. Braun, and Marius Kloft. 2015. Hidden Markov Anomaly Detection. In *ICML (JMLR Workshop and Conference Proceedings, Vol. 37)*. JMLR.org, 1833–1842.
- [23] John C Gower and Gavin JS Ross. 1969. Minimum spanning trees and single linkage cluster analysis. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 18, 1 (1969), 54–64.
- [24] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. 2017. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In *ICWS*. IEEE, 33–40.
- [25] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. 2021. A Survey on Automated Log Analysis for Reliability Engineering. *ACM Comput. Surv.* 54, 6 (2021), 130:1–130:37.
- [26] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. 2022. A Survey on Automated Log Analysis for Reliability Engineering. *ACM Comput. Surv.* 54, 6 (2022), 130:1–130:37.
- [27] Shilin He, Qingwei Lin, Jian-Guang Lou, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. 2018. Identifying impactful service system problems via log analysis. In *ESEC/SIGSOFT FSE*. ACM, 60–70.
- [28] Shilin He, Xu Zhang, Pinjia He, Yong Xu, Liqun Li, Yu Kang, Minghua Ma, Yining Wei, Yingnong Dang, Saravanakumar Rajmohan, and Qingwei Lin. 2022. An empirical study of log analysis at Microsoft. In *ESEC/SIGSOFT FSE*. ACM, 1465–1476.
- [29] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. 2016. Experience Report: System Log Analysis for Anomaly Detection. In *ISSRE*. IEEE Computer Society, 207–218.
- [30] Joerd Hiemstra. 2000. A probabilistic justification for using tf x idf term weighting in information retrieval. *Int. J. Digit. Libr.* 3, 2 (2000), 131–139.
- [31] Qiao Huang, Xin Xia, and David Lo. 2017. Supervised vs Unsupervised Models: A Holistic Look at Effort-Aware Just-in-Time Defect Prediction. In *ICSME*. IEEE Computer Society, 159–170.
- [32] J Edward Jackson and Govind S Mudholkar. 1979. Control procedures for residuals associated with principal component analysis. *Technometrics* 21, 3 (1979), 341–349.
- [33] J Edward Jackson and Govind S Mudholkar. 1979. Control procedures for residuals associated with principal component analysis. *Technometrics* 21, 3 (1979), 341–349.
- [34] Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Herve Jégou, and Tomas Mikolov. 2016. FastText.zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651* (2016).
- [35] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomás Mikolov. 2017. Bag of Tricks for Efficient Text Classification. In *EACL (2)*. Association for Computational Linguistics, 427–431.
- [36] Van-Hoang Le and Hongyu Zhang. 2021. Log-based Anomaly Detection Without Log Parsing. In *ASE*. IEEE, 492–504.
- [37] Van-Hoang Le and Hongyu Zhang. 2022. Log-based Anomaly Detection with Deep Learning: How Far Are We? *CoRR* abs/2202.04301 (2022).
- [38] Richard C. T. Lee, Y. H. Chin, and S. C. Chang. 1976. Application of Principal Component Analysis to Multikey Searching. *IEEE Trans. Software Eng.* 2, 3 (1976), 185–193.

- [39] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Lei Qin Yan, Zikai Wang, Zhekang Chen, Wenchi Zhang, Xiaohui Nie, Kaixin Sui, and Dan Pei. 2021. Practical Root Cause Localization for Microservice Systems via Trace Analysis. In *IWQoS*. IEEE, 1–10.
- [40] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. 2016. Log clustering based problem identification for online service systems. In *ICSE (Companion Volume)*. ACM, 102–111.
- [41] Jinyang Liu, Jieming Zhu, Shilin He, Pinjia He, Zibin Zheng, and Michael R. Lyu. 2019. Logzip: Extracting Hidden Structures via Iterative Clustering for Log Compression. In *ASE*. IEEE, 863–873.
- [42] Jian-Guang Lou, Qiang Fu, Shengqi Yang, Ye Xu, and Jiang Li. 2010. Mining Invariants from Console Logs for System Problem Detection. In *USENIX Annual Technical Conference*. USENIX Association.
- [43] Siyang Lu, Xiang Wei, Yandong Li, and Liqiang Wang. 2018. Detecting Anomaly in Big Data System Logs Using Convolutional Neural Network. In *DASC/PiCom/DataCom/CyberSciTech*. IEEE Computer Society, 151–158.
- [44] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to information retrieval*. Cambridge University Press.
- [45] Leland McInnes and John Healy. 2017. Accelerated Hierarchical Density Based Clustering. In *ICDM Workshops*. IEEE Computer Society, 33–42.
- [46] Weibin Meng, Ying Liu, Yuheng Huang, Shenglin Zhang, Federico Zaiter, Bingjin Chen, and Dan Pei. 2020. A Semantic-aware Representation Framework for Online Log Analysis. In *ICCCN*. IEEE, 1–7.
- [47] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *IJCAI*. ijcai.org, 4739–4745.
- [48] Meiyappan Nagappan and Mladen A. Vouk. 2010. Abstracting log lines to log event types for mining software system logs. In *MSR*. IEEE Computer Society, 114–117.
- [49] Kim Anh Nguyen, Sabine Schulte im Walde, and Ngoc Thang Vu. 2016. Integrating Distributional Lexical Contrast into Word Embeddings for Antonym-Synonym Distinction. In *ACL (2)*. The Association for Computer Linguistics.
- [50] Adam J. Oliner and Jon Stearley. 2007. What Supercomputers Say: A Study of Five System Logs. In *DSN*. IEEE Computer Society, 575–584.
- [51] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski, and Odej Kao. 2021. Robust and Transferable Anomaly Detection in Log Data using Pre-Trained Language Models. *CoRR* abs/2102.11570 (2021).
- [52] Randy C. Paffenroth, Kathleen Kay, and Les Servi. 2018. Robust PCA for Anomaly Detection in Cyber Networks. *CoRR* abs/1801.01571 (2018).
- [53] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Empirical Methods in Natural Language Processing (EMNLP)*. 1532–1543. <http://www.aclweb.org/anthology/D14-1162>
- [54] Justyna Petke, Myra B. Cohen, Mark Harman, and Shin Yoo. 2015. Practical Combinatorial Interaction Testing: Empirical Findings on Efficiency and Early Fault Detection. *IEEE Trans. Software Eng.* 41, 9 (2015), 901–924.
- [55] Samira Pouyanfar, Saad Sadiq, Yilin Yan, Haiman Tian, Yudong Tao, Maria Presa Reyes, Mei-Ling Shyu, Shu-Ching Chen, and Sundaraja S Iyengar. 2018. A survey on deep learning: Algorithms, techniques, and applications. *ACM Computing Surveys (CSUR)* 51, 5 (2018), 1–36.
- [56] David M. W. Powers. 2020. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *CoRR* abs/2010.16061 (2020).
- [57] C Radhakrishna Rao. 1964. The use and interpretation of principal component analysis in applied research. *Sankhyā: The Indian Journal of Statistics, Series A* (1964), 329–358.
- [58] Brian D Ripley. 2007. *Pattern recognition and neural networks*. Cambridge university press.
- [59] Gerard Salton and Chris Buckley. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Inf. Process. Manag.* 24, 5 (1988), 513–523.
- [60] Ajay Shrestha and Ausif Mahmood. 2019. Review of deep learning algorithms and architectures. *IEEE access* 7 (2019), 53040–53065.
- [61] Mei-Ling Shyu, Shu-Ching Chen, Kanoksri Sarinnapakorn, and LiWu Chang. 2003. *A novel anomaly detection scheme based on principal component classifier*. Technical Report. Miami Univ Coral Gables FL Dept of Electrical and Computer Engineering.
- [62] Jon Stearley and Adam J. Oliner. 2008. Bad Words: Finding Faults in Spirit’s Syslogs. In *CCGRID*. IEEE Computer Society, 765–770.
- [63] Martin Sundermeyer, Hermann Ney, and Ralf Schlüter. 2015. From Feedforward to Recurrent LSTM Neural Networks for Language Modeling. *IEEE ACM Trans. Audio Speech Lang. Process.* 23, 3, 517–529.
- [64] Byungchul Tak and Wook-Shin Han. 2021. Lognroll: discovering accurate log templates by iterative filtering. In *Middleware*. ACM, 273–285.
- [65] Naoya Takeishi. 2019. Shapley Values of Reconstruction Errors of PCA for Explaining Anomaly Detection. (2019), 793–798.

- [66] Feifei Tu, Jiaxin Zhu, Qimu Zheng, and Minghui Zhou. 2018. Be careful of when: an empirical study on time-related misuse of issue tracking data. In *ESEC/SIGSOFT FSE*. ACM, 307–318.
- [67] Risto Vaarandi. 2003. A data clustering algorithm for mining patterns from event logs. In *Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM 2003)(IEEE Cat. No. 03EX764)*. Ieee, 119–126.
- [68] Risto Vaarandi and Mauno Pihelgas. 2015. LogCluster - A data clustering and pattern mining algorithm for event logs. In *CNSM*. IEEE Computer Society, 1–7.
- [69] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. 2017. Long short-term memory based operation log anomaly detection. In *ICACCI*. IEEE, 236–242.
- [70] Lingzhi Wang, Nengwen Zhao, Junjie Chen, Pinnong Li, Wenchi Zhang, and Kaixin Sui. 2020. Root-Cause Metric Location for Microservice Systems via Log Anomaly Detection. In *ICWS*. IEEE, 142–150.
- [71] Weijing Wang, Junjie Chen, Lin Yang, Hongyu Zhang, and Zan Wang. 2023. Understanding and predicting incident mitigation time. *Inf. Softw. Technol.* 155 (2023), 107119.
- [72] Weijing Wang, Junjie Chen, Lin Yang, Hongyu Zhang, Pu Zhao, Bo Qiao, Yu Kang, Qingwei Lin, Saravanakumar Rajmohan, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2021. How Long Will it Take to Mitigate this Incident for Online Service Systems?. In *ISSRE*. IEEE, 36–46.
- [73] Zan Wang, Hanmo You, Junjie Chen, Yingyi Zhang, Xuyuan Dong, and Wenbin Zhang. 2021. Prioritizing Test Inputs for Deep Neural Networks via Mutation Analysis. In *ICSE*. IEEE, 397–409.
- [74] Bin Xia, Junjie Yin, Jian Xu, and Yun Li. 2019. LogGAN: A Sequence-Based Generative Adversarial Network for Anomaly Detection Based on System Logs. In *SciSec (Lecture Notes in Computer Science, Vol. 11933)*. Springer, 61–76.
- [75] Yongzheng Xie, Hongyu Zhang, and Muhammad Ali Babar. 2022. LogGD: Detecting Anomalies from System Logs by Graph Neural Networks. *CoRR* abs/2209.07869 (2022).
- [76] Wei Xu, Ling Huang, Armando Fox, David A. Patterson, and Michael I. Jordan. 2010. Detecting Large-Scale System Problems by Mining Console Logs. In *ICML*. Omnipress, 37–46.
- [77] Wei Xu, Ling Huang, Armando Fox, David A. Patterson, and Michael I. Jordan. 2010. Detecting Large-Scale System Problems by Mining Console Logs. In *ICML*. Omnipress, 37–46.
- [78] Kenji Yamanishi and Yuko Maruyama. 2005. Dynamic syslog mining for network failure monitoring. In *KDD*. ACM, 499–508.
- [79] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Semi-supervised Log-based Anomaly Detection via Probabilistic Label Estimation. In *ICSE*. IEEE, 1448–1460.
- [80] Danfeng (Daphne) Yao, Xiaokui Shu, Long Cheng, and Salvatore J. Stolfo. 2017. *Anomaly Detection as a Service: Challenges, Advances, and Opportunities*. Morgan & Claypool Publishers.
- [81] Bo Zhang, Hongyu Zhang, Pablo Moscato, and Aozhong Zhang. 2020. Anomaly Detection via Mining Numerical Workflow Relations from Logs. In *SRDS*. IEEE, 195–204.
- [82] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-based Deep Learning. In *ICSE*. ACM, 623–634.
- [83] Shengming Zhang, Yanchi Liu, Xuchao Zhang, Wei Cheng, Haifeng Chen, and Hui Xiong. 2022. CAT: Beyond Efficient Transformer for Content-Aware Anomaly Detection in Event Sequences. In *KDD*. ACM, 4541–4550.
- [84] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xincheng Yang, Qian Cheng, Ze Li, Junjie Chen, Xiaoting He, Randolph Yao, Jian-Guang Lou, Murali Chintalapati, Furao Shen, and Dongmei Zhang. 2019. Robust log-based anomaly detection on unstable log data. In *ESEC/SIGSOFT FSE*. ACM, 807–817.
- [85] Yanyong Zhang and Anand Sivasubramaniam. 2008. Failure prediction in IBM BlueGene/L event logs. In *IPDPS*. IEEE, 1–5.
- [86] Yingyi Zhang, Zan Wang, Jiajun Jiang, Hanmo You, and Junjie Chen. 2022. Toward Improving the Robustness of Deep Learning Models via Model Transformation. In *ASE*. ACM, 104:1–104:13.
- [87] Nengwen Zhao, Junjie Chen, Zhaoyang Yu, Honglin Wang, Jiesong Li, Bin Qiu, Hongyu Xu, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2021. Identifying bad software changes via multimodal anomaly detection for online service systems. In *ESEC/SIGSOFT FSE*. ACM, 527–539.
- [88] Nengwen Zhao, Honglin Wang, Zeyan Li, Xiao Peng, Gang Wang, Zhu Pan, Yong Wu, Zhen Feng, Xidao Wen, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2021. An empirical investigation of practical log anomaly detection for online service systems. In *ESEC/SIGSOFT FSE*. ACM, 1404–1415.
- [89] Zhenfei Zhao, Weina Niu, Xiaosong Zhang, Runzi Zhang, Zhenqi Yu, and Cheng Huang. 2021. Trine: Syslog anomaly detection with three transformer encoders in one generative adversarial network. *Applied Intelligence* (2021), 1–10.
- [90] Junwei Zhou, Yijia Qian, Qingtian Zou, Peng Liu, and Jianwen Xiang. 2022. DeepSyslog: Deep Anomaly Detection on Syslog Using Sentence Embedding and Metadata. *IEEE Trans. Inf. Forensics Secur.* 17 (2022), 3051–3061.
- [91] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *ESEC/SIGSOFT FSE*. ACM,

683–694.

- [92] Jieming Zhu, Pinjia He, Qiang Fu, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. 2015. Learning to Log: Helping Developers Make Informed Logging Decisions. In *ICSE (1)*. IEEE Computer Society, 415–425.
- [93] Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R. Lyu. 2019. Tools and benchmarks for automated log parsing. In *ICSE (SEIP)*. IEEE / ACM, 121–130.